CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

PROJECT REPORT

STUDENT MANAGEMENT SYSTEM IN C++

Course & Code:	Object Oriented Programming (25CAH-201)
Submitted By:	
Name:	Rakshak
UID:	25BCD10102
Branch	BCA (Data Science)
Section:	25BCD2(A)
Semester:	Semester 3
Submitted To:	
TEACHAR NAME:	Mr. Jitendra Kumar
DESIGNATION:	Professor

SIGN :- _____

1. Introduction

Managing student information is an essential operational requirement in academic institutions including schools, colleges, and training academies. Accurate handling of student attributes—such as unique identification numbers, student names, age, enrolled courses, and academic scores—is paramount to administrative efficiency and academic evaluation. Conventional manual or physical record-keeping is notoriously labor-intensive, error-prone, susceptible to duplicate records, and inefficient when querying or modifying data.

This project report presents the design and implementation of a console-based Student Management System developed in C++. The software leverages foundational Object-Oriented Programming (OOP) paradigms to encapsulate student records, manipulate data collections using the C++ Standard Template Library (STL vector), and enforce persistence via file streams (fstream) without requiring an external database management system.

2. Problem Statement

Educational institutions often require a lightweight, dependable, and self-contained software utility to maintain student academic records. Commercial database engines frequently introduce unwanted setup overhead, licensing costs, and operational complexity for single-terminal desktop use. Consequently, the objective is to engineer a standalone, menu-driven C++ application capable of performing standard CRUD (Create, Read, Update, Delete) operations, calculating academic grades, and maintaining state across sessions through local file handling.

3. Project Objectives

- Design and deploy an interactive, menu-driven console application in standard C++17.
- Model real-world entities through Object-Oriented Programming principles (Student and StudentManager classes).
- Implement robust data encapsulation by maintaining student attributes as private class members.
- Utilize C++ Standard Template Library (STL) vector to manage dynamic in-memory collections of student records.
- Ensure persistent storage between application lifecycles using text-based file streams (fstream).
- Prevent duplicate ID assignment and enforce comprehensive input validation across user entry points.
- Automate grade evaluation from percentage marks based on a predefined institutional rubric.

4. Scope of the Project

The Student Management System is targeted for single-terminal, local desktop environments. The system encapsulates student attributes including Student ID, Full Name, Age, Course Enrollment, and Aggregate Marks. Core functional capabilities include record creation, formatted tabular display, unique ID querying, record modification, record deletion, and programmatic exit.

Deliberate Architectural Boundaries: The current release intentionally avoids client-server architecture, multi-user concurrency locks, cryptographic password authorization, and graphical user interfaces (GUI). It prioritizes educational demonstration, algorithmic clarity, code maintainability, and clean OOP abstractions.

5. Tools and Technologies

Tool / Technology	Description & Project Role
C++ (C++17 Standard)	Core programming language chosen for performance, type safety, and modern standard library features.
VS Code	Integrated Development Environment utilized for source code editing, syntax highlighting, and debugging.
MSYS2 / MinGW g++	Underlying C++ compiler toolchain used to build and compile C++17 compliant binaries.
OOP Classes & Objects	Encapsulates entity attributes into Student and business logic/state management into StudentManager.
STL Vector (std::vector)	Dynamic sequential container providing amortized O(1) insertion and flexible record handling.
File Streams (std::fstream)	Handles persistent disk I/O, writing formatted strings to and deserializing records from students.txt.
students.txt	Delimited flat text file acting as the lightweight persistent datastore.

6. System Architecture and Design

6.1 Student Class (Data Entity)

The Student class encapsulates attributes for individual students. By applying encapsulation, the internal fields (id, name, age, course, marks) are kept strictly private. Public accessor and utility member

functions expose safe interfaces to retrieve the student ID, evaluate and return letter grades, format student data for console output, and serialize records to disk.

6.2 StudentManager Class (Controller / Service Layer)

The StudentManager class acts as the operational coordinator. It holds an internal std::vector<Student> collection and coordinates end-user requests with storage operations. Key responsibilities include:

- Linear ID search via findIndex() to guarantee uniqueness and verify existence.
- Startup deserialization (loadAll()) to parse records from students.txt into memory.
- Synchronous persistence (saveAll()) executing post-mutation writes upon add, update, or delete operations.
- Type-safe console input wrappers (readInt, readMarks, readText) mitigating stream failures and buffer issues.

6.3 Program Workflow

1. Application Initialization: Construct StudentManager, invoke loadAll(), and populate the vector from students.txt.

2. Main Menu Render: Present interactive terminal options (1: Add, 2: Display, 3: Search, 4: Update, 5: Delete, 0: Exit).

3. Input Validation & Dispatch: Capture user selection, clean input streams, and invoke corresponding handler methods.

4. Transaction Commit: If an update, creation, or deletion succeeds, immediately serialize updated records to students.txt.

5. Loop & Termination: Re-render the menu until the user triggers option 0, printing a clean termination message.

7. File Storage Format

Persistence is established via a flat text file named students.txt. Records are written one per line using a pipe-delimited (|) schema:

```
Schema: <ID>|<Name>|<Age>|<Course>|<Marks>

Example Data:
101|Aman Sharma|19|BCA|85.00
102|Priya Singh|20|BCA Data Science|92.00
```

The delimiter character '|' is strictly forbidden in string inputs (Name and Course) to prevent schema corruption during deserialization.

8. Main Features

System Feature	Operational Description
Add Student	Validates ID uniqueness and positive ranges; prompts for student fields and commits record to memory and disk.
Display All Students	Formats student records into aligned tabular columns showing ID, Name, Age, Course, Marks, and Grade.
Search Student	Queries vector for an ID; renders matching record in full tabular detail or notifies user if absent.
Update Student	Prompts for revised attributes of an existing ID while retaining primary ID integrity; saves state immediately.
Delete Student	Removes matching student record from collection using vector iterators and synchronizes the flat file.
Grade Calculation	Dynamically evaluates grade brackets: A+ (>=90), A (>=80), B (>=70), C (>=60), D (>=50), F (<50).
Persistent Storage	Executes silent automatic loading on startup and atomic record dumping upon state changes.

9. Input Validation and Assumptions

- **Primary Key Integrity:** Student IDs must be strictly positive whole numbers (> 0) and unique across the database.
- **Age Boundaries:** Accepted age range is constrained between 1 and 120 years.
- **Score Validation:** Marks are bounded between 0.00 and 100.00 inclusive.
- **String Sanitization:** Names and Course names cannot be blank and must not include the '|' pipe character.
- **Execution Environment:** The program assumes write permissions within the active working directory for students.txt creation.

10. Verification and Testing Plan

Test Case	Test Input / Action	Expected Result
TC-01: Add Valid Record	Enter ID 105, 'David Lee', Age 20, 'History', 79.5	Record appended to memory and written to students.txt

TC-02: Duplicate ID Prevention	Enter duplicate ID 101 with new record data	Rejection message displayed: 'Student ID already exists.'
TC-03: Display Formatted List	Select option 2 (Display All Students)	Formatted table rendered with aligned columns and grades
TC-04: Successful Search	Query existing ID 102	Outputs matching record with calculated grade (A+)
TC-05: Search Miss	Query non-existent ID 999	Displays error message: 'Student not found.'
TC-06: Record Update	Update record 103 with updated marks/course	Vector element replaced and students.txt rewritten
TC-07: Record Deletion	Delete ID 104 from system	Vector erased at index; disk file updated with 1 fewer row
TC-08: Cold Boot Persistence	Terminate executable, re-launch application	All saved records reloaded into vector during loadAll()

11. Complete Source Code Implementation

The complete C++17 implementation code is provided below. The program is self-contained in main.cpp and compiles directly using any standard C++17 compiler (e.g., g++ -std=c++17 main.cpp -o sms.exe).

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <limits>
#include <iomanip>

using namespace std;

const string FILE_NAME = "students.txt";

class Student {
private:
    int id;
    string name;
    int age;
    string course;
    double marks;

public:
    Student(int i, string n, int a, string c, double m)
        : id(i), name(n), age(a), course(c), marks(m) {}
```

```
    int getId() const {
        return id;
    }

    string grade() const {
        if (marks >= 90) return "A+";
        if (marks >= 80) return "A";
        if (marks >= 70) return "B";
        if (marks >= 60) return "C";
        if (marks >= 50) return "D";
        return "F";
    }

    void display() const {
        cout << left
             << setw(8) << id
             << setw(22) << name
             << setw(8) << age
             << setw(18) << course
             << setw(10) << fixed << setprecision(2) << marks
             << grade() << '\n';
    }

    void save(ofstream& file) const {
        file << id << '|' << name << '|' << age << '|'
             << course << '|' << marks << '\n';
    }
};

class StudentManager {
private:
    vector<Student> students;

    int findIndex(int id) const {
        for (int i = 0; i < static_cast<int>(students.size()); i++) {
            if (students[i].getId() == id)
                return i;
        }
        return -1;
    }

    void saveAll() const {
        ofstream file(FILE_NAME);
        if (!file) {
            cout << "Error: Could not save the file.\n";
            return;
        }
        for (const Student& s : students)
            s.save(file);
    }

    void loadAll() {
        ifstream file(FILE_NAME);
        string line;

        while (getline(file, line)) {
            stringstream ss(line);
            string idText, name, ageText, course, marksText;

            if (!getline(ss, idText, '|') ||
```

```
            !getline(ss, name, '|') ||
            !getline(ss, ageText, '|') ||
            !getline(ss, course, '|') ||
            !getline(ss, marksText))
            continue;

        try {
            int id = stoi(idText);
            int age = stoi(ageText);
            double marks = stod(marksText);

            if (id > 0 && age > 0 &&
                marks >= 0 && marks <= 100 &&
                findIndex(id) == -1) {
                students.emplace_back(id, name, age, course, marks);
            }
        } catch (...) {
            // Ignore malformed records.
        }
    }
}

static int readInt(const string& prompt) {
    int value;
    while (true) {
        cout << prompt;
        if (cin >> value) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Please enter a valid whole number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

static double readMarks() {
    double marks;
    while (true) {
        cout << "Enter marks (0-100): ";
        if (cin >> marks && marks >= 0 && marks <= 100) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return marks;
        }
        cout << "Invalid marks. Enter a value from 0 to 100.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

static string readText(const string& prompt) {
    string value;
    while (true) {
        cout << prompt;
        getline(cin, value);
        if (!value.empty() && value.find('|') == string::npos)
            return value;
        cout << "Input cannot be empty or contain '|'.\n";
    }
}
```

```
    void printHeader() const {
        cout << left
             << setw(8) << "ID"
             << setw(22) << "Name"
             << setw(8) << "Age"
             << setw(18) << "Course"
             << setw(10) << "Marks"
             << "Grade\n";
        cout << string(76, '-') << '\n';
    }

public:
    StudentManager() {
        loadAll();
    }

    void addStudent() {
        cout << "\n--- Add Student ---\n";
        int id = readInt("Enter student ID: ");

        if (id <= 0) {
            cout << "ID must be positive.\n";
            return;
        }
        if (findIndex(id) != -1) {
            cout << "Student ID already exists.\n";
            return;
        }

        string name = readText("Enter name: ");
        int age = readInt("Enter age: ");
        if (age <= 0 || age > 120) {
            cout << "Please enter a valid age.\n";
            return;
        }

        string course = readText("Enter course: ");
        double marks = readMarks();

        students.emplace_back(id, name, age, course, marks);
        saveAll();
        cout << "Student added successfully!\n";
    }

    void displayStudents() const {
        cout << "\n--- All Students ---\n";
        if (students.empty()) {
            cout << "No student records found.\n";
            return;
        }
        printHeader();
        for (const Student& s : students)
            s.display();
    }

    void searchStudent() const {
        int id = readInt("Enter student ID to search: ");
        int index = findIndex(id);
        if (index == -1) {
```

```
            cout << "Student not found.\n";
            return;
        }
        printHeader();
        students[index].display();
    }

    void updateStudent() {
        int id = readInt("Enter student ID to update: ");
        int index = findIndex(id);
        if (index == -1) {
            cout << "Student not found.\n";
            return;
        }

        cout << "Enter the student's updated details:\n";
        string name = readText("Enter new name: ");
        int age = readInt("Enter new age: ");
        if (age <= 0 || age > 120) {
            cout << "Invalid age. Update cancelled.\n";
            return;
        }

        string course = readText("Enter new course: ");
        double marks = readMarks();

        students[index] = Student(id, name, age, course, marks);
        saveAll();
        cout << "Student updated successfully!\n";
    }

    void deleteStudent() {
        int id = readInt("Enter student ID to delete: ");
        int index = findIndex(id);
        if (index == -1) {
            cout << "Student not found.\n";
            return;
        }

        students.erase(students.begin() + index);
        saveAll();
        cout << "Student deleted successfully!\n";
    }

    void run() {
        int choice;
        do {
            cout << "\n================================\n";
            cout << "   STUDENT MANAGEMENT SYSTEM\n";
            cout << "================================\n";
            cout << "1. Add Student\n";
            cout << "2. Display All Students\n";
            cout << "3. Search Student\n";
            cout << "4. Update Student\n";
            cout << "5. Delete Student\n";
            cout << "0. Exit\n";
            cout << "================================\n";

            choice = readInt("Enter your choice: ");
            switch (choice) {
```

```
                case 1: addStudent(); break;
                case 2: displayStudents(); break;
                case 3: searchStudent(); break;
                case 4: updateStudent(); break;
                case 5: deleteStudent(); break;
                case 0:
                    cout << "Thank you for using the system!\n";
                    break;
                default:
                    cout << "Invalid choice. Try again.\n";
            }
        } while (choice != 0);
    }
};

int main() {
    StudentManager manager;
    manager.run();
    return 0;
}
```

12. Sample Terminal Execution Output

The following terminal transcript demonstrates sample execution runs of adding students, displaying all records in formatted columns, and menu navigation:

```
================================
   STUDENT MANAGEMENT SYSTEM
================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
0. Exit
================================
Enter your choice: 2

--- All Students ---
ID      Name                 Age     Course           Marks     Grade
-------------------------------------------------------------------------
101     Arun Sharma          20      Computer Science  85.50     A
102     Jane Smith           21      Mathematics       92.00     A+
103     Michael Chang        19      Physics           74.00     B
104     Priya Patel          22      Chemistry         68.50     C

================================
   STUDENT MANAGEMENT SYSTEM
================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
0. Exit
================================
```

```
Enter your choice: 1

--- Add Student ---
Enter student ID: 105
Enter name: David Lee
Enter age: 20
Enter course: History
Enter marks (0-100): 79.5
Student added successfully!

================================
   STUDENT MANAGEMENT SYSTEM
================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
0. Exit
================================
Enter your choice: _
```

13. Expected Outcome

The expected deliverable is a dependable console software suite capable of managing student educational records without software regression or data loss across restart cycles. The project successfully proves the application of OOP concepts, STL memory management, and file stream handling to solve everyday educational administration problems.

14. Limitations and Future Scope

While the current implementation fulfills all core educational requirements, future iterations can introduce production-grade features:

- **Modular Architecture:** Separate classes into distinct header files (.h/.hpp) and implementation files (.cpp) with CMake/Makefile build automation.
- **Advanced Analytics & Reporting:** Introduce class rank computation, GPA translation, cohort averages, filtering by course, and automated grade distribution histograms.
- **Relational Database Integration:** Migrate flat file storage to embedded SQLite or MySQL for ACID compliance, transactions, and index acceleration.
- **Concurrency & File Locking:** Implement file stream mutex locking to allow safe simultaneous instances.
- **Security & Access Control:** Incorporate role-based access control (Admin, Instructor, Student) with hashed credential authentication.

15. Conclusion

The Student Management System successfully showcases the power and clarity of modern C++ Object-Oriented Programming for real-world record administration. By combining encapsulation, dynamic STL storage, defensive stream validation, and delimiter-based file persistence, the project delivers a clean, modular foundation that satisfies all academic specifications while providing clear paths for future enterprise enhancements.